

Kotlin 密封类教学

Kotlin密封类教学讲义（零基础入门）

各位同学，今天咱们专门聊聊Kotlin中的**密封类（Sealed Class）**。它是Kotlin的“特色武器”，能帮我们**安全地处理“有限且明确的分支场景”**（比如网络请求结果只有“成功、失败、加载中”三种，或者交通灯只有“红、黄、绿”三种状态）。这节课用“生活案例+代码对比”的方式，讲清它的价值、原理和用法！

一、先搞懂：为什么需要密封类？

想象一个场景：你开发了一个“处理交通灯”的功能，交通灯只有**红、黄、绿**三种状态。如果用普通的 `when` 判断，代码会有隐患：

Code block

```
1  // 普通类模拟交通灯状态
2  open class TrafficLight
3  class Red : TrafficLight()
4  class Yellow : TrafficLight()
5  class Green : TrafficLight()
6
7  fun handleLight(light: TrafficLight) {
8      when (light) {
9          is Red -> println("红灯停")
10         is Yellow -> println("黄灯等")
11         is Green -> println("绿灯行")
12         // 这里编译器不会提醒你遗漏分支！
13         // 万一后来加了个Blue灯，你很容易忘记在这里加逻辑，导致Bug
14     }
15 }
```

这就是**密封类要解决的核心问题**：当分支是“有限且明确”时，让编译器帮你检查“是否穷尽了所有可能”，避免遗漏分支导致的逻辑错误。

二、密封类的核心价值：“穷尽所有可能”的安全保障

密封类的本质是**“受限制的继承”——它的子类必须和它定义在同一个文件中**，这样编译器就能明确知道“所有可能的子类”，从而在 `when` 判断时提醒你是否遗漏分支。

用密封类重构交通灯案例：

Code block

```
1  // 密封类：交通灯（子类必须和它在同一个文件）
2  sealed class TrafficLightSealed
3
4  // 密封类的子类（必须在同一个文件）
5  object Red : TrafficLightSealed()
6  object Yellow : TrafficLightSealed()
7  object Green : TrafficLightSealed()
8
9  fun handleSealedLight(light: TrafficLightSealed) {
10     when (light) {
11         is Red -> println("红灯停")
12         is Yellow -> println("黄灯等")
13         is Green -> println("绿灯行")
14         // 这里如果少写一个分支，编译器会直接报错！
15     }
16 }
```

效果：如果后来有人想加一个 `Blue` 灯，必须在同一个文件中定义它的子类，此时 `when` 判断会立即报错，提醒你补充逻辑——这就从根源上避免了“遗漏分支”的Bug。

三、其他语言有对应的东西吗？

Kotlin的密封类是**“类版本的枚举”**，但比枚举更灵活（枚举是单例，密封类的子类可以是普通类、数据类，甚至带属性和方法）。

- **Java**: 没有完全对应的特性，但可以通过“包私有类+工具类”模拟（麻烦且不优雅）；
- **Scala**: 有 `sealed trait`，和Kotlin的密封类思路一致；
- **Swift**: 有 `enum` 的“关联值”特性，也能实现类似的“有限分支安全判断”。

四、密封类的典型使用场景

场景1：网络请求结果（成功、失败、加载中）

Code block

```
1  // 密封类：网络请求结果
2  sealed class Result<out T> {
3      data class Success<out T>(val data: T) : Result<T>()
4      data class Error(val message: String) : Result<Nothing>()
5      object Loading : Result<Nothing>()
6  }
7
8  // 处理请求结果
9  fun <T> handleResult(result: Result<T>) {
10     when (result) {
11         is Result.Success -> println("请求成功，数据: ${result.data}")
12         is Result.Error -> println("请求失败，原因: ${result.message}")
13         Result.Loading -> println("请求加载中...")
14     }
15 }
16
17 fun main() {
18     val success = Result.Success("用户列表数据")
19     val error = Result.Error("网络连接超时")
20     val loading = Result.Loading
21
22     handleResult(success)
23     handleResult(error)
24     handleResult(loading)
25 }
```

场景2：UI状态（内容、空页面、错误页）

Code block

```
1  // 密封类: UI状态
2  sealed class UiState {
3      data class Content(val data: List<String>) : UiState()
4      object Empty : UiState()
5      data class Error(val errorMsg: String) : UiState()
6  }
7
8  // 渲染UI
9  fun renderUi(state: UiState) {
10     when (state) {
11         is UiState.Content -> println("显示内容: ${state.data}")
12         UiState.Empty -> println("显示空页面")
13         is UiState.Error -> println("显示错误: ${state.errorMsg}")
14     }
15 }
```

五、密封类的使用规则

1. **定义位置**: 密封类的子类必须和密封类定义在**同一个文件中**（Kotlin 1.5+支持在同一模块的不同文件，但建议还是放在同一个文件更直观）；
2. **子类类型**: 子类可以是 `object`（单例）、`data class`（数据类）、普通 `class`（带属性和方法）；
3. **when 判断**: 在 `when` 中使用密封类时，无需写 `else` 分支，编译器会检查是否穷尽所有子类。

总结与作业

核心知识点

- 密封类解决的问题：确保“有限分支场景”的逻辑完整性，避免 `when` 判断遗漏分支；
- 核心特性：子类必须和密封类在同一个文件，编译器可检查分支穷尽性；
- 典型场景：网络请求结果、UI状态、有限状态机（如交通灯、订单状态）。

课后作业

定义一个密封类 `OrderStatus`，包含以下子类：

- `Pending`（待支付）
- `Paid`（已支付，带支付时间 `String`）
- `Shipped`（已发货，带物流单号 `String`）
- `Delivered`（已送达）

然后写一个 `handleOrder` 函数，用 `when` 判断所有状态并打印对应信息，体验密封类的“分支穷尽检查”功能。

试着自己写代码实现，有问题随时问哦！

（注：文档部分内容可能由 AI 生成）